LAB SVM

65090502417 ปิยพัทธ์ จันทร์ทวี

22/09/2568

1.) kfold

2.) pre rec f1

```
library(dplyr)
```

```
##
## Attaching package: 'dplyr'
```

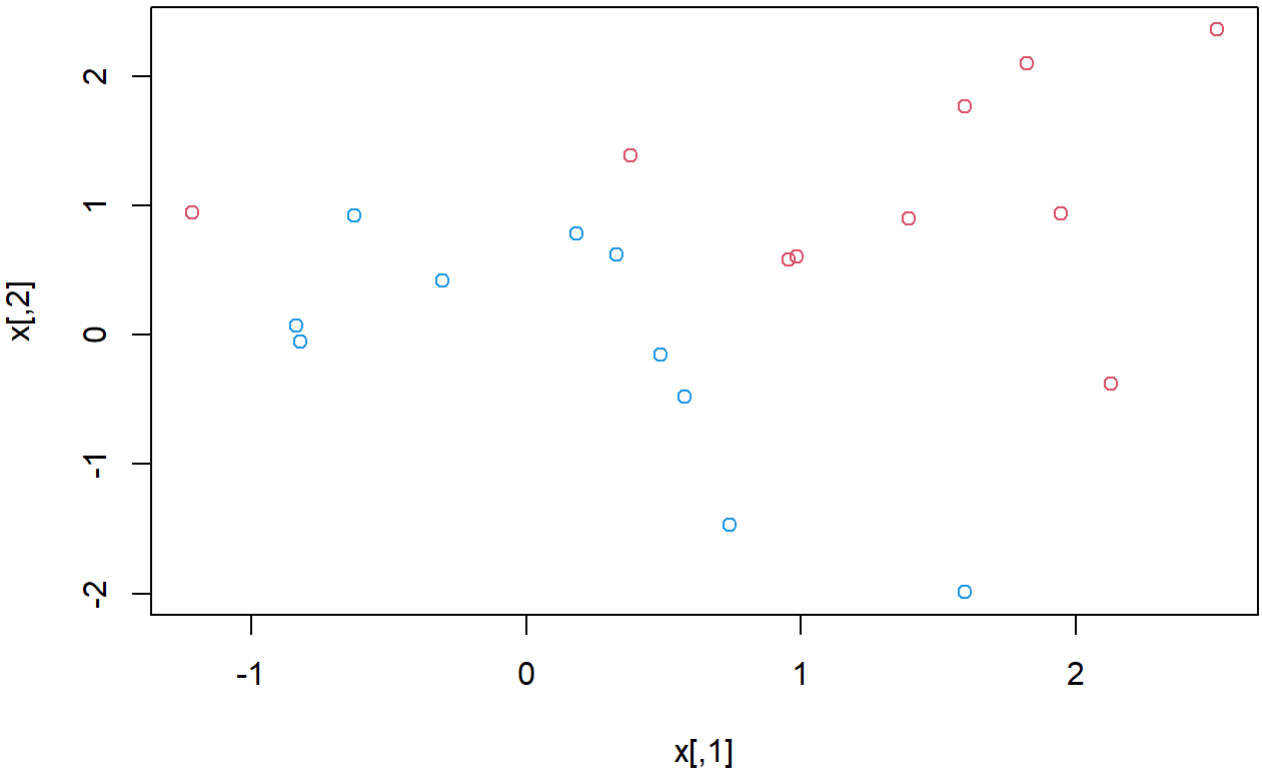
```
## The following objects are masked from 'package:stats':
##
##     filter, lag
```

```
## The following objects are masked from 'package:base':
##
##     intersect, setdiff, setequal, union
```

```
library(e1071)
```

```
## Warning: package 'e1071' was built under R version 4.4.3
```

```
set.seed(1)
x <- matrix(rnorm(20 * 2), ncol = 2)
y <- c(rep(-1, 10), rep(1, 10))
x[y == 1, ] <- x[y == 1, ] + 1
plot(x, col = (3 - y))
```


x[,2]
2
1
0
-1
-2
-1
0
1
2
x[,1]

```
###
dat <- data.frame(x = x, y = as.factor(y))

K=10
set.seed(123)
folds=sample(1:K,nrow(dat),replace=TRUE)

acc <- 1:K
pre <- 1:K
rec <- 1:K
f1 <- 1:K
tunecost <- 1:K
cost_tune <-  0.1^c(seq(-1, 1, by = 0.01))

results <- data.frame(
  cost = cost_tune,
  acc = NA,
  pre = NA,
  rec = NA,
  f1 = NA
)

for (s in 1:length(cost_tune)){
  for (i in 1:K){
    traindat <- folds!=i

    svmfit <- svm(y ~ ., data = dat[traindat,], kernel = "linear",
                cost = cost_tune[s], scale = FALSE)

    ypred <- predict(svmfit, dat[-traindat,])
    mod_tab <- table(ypred, dat[-traindat,]$y)

    acc[i] <-  (mod_tab[1,1] + mod_tab[2,2])/((mod_tab[1,1] + mod_tab[2,2]) + (mod_tab[2,1] +
mod_tab[1,2]))
    pre[i] <- mod_tab[2,2]/(mod_tab[2,2] + mod_tab[2,1])
    rec[i] <- mod_tab[2,2]/(mod_tab[2,2] + mod_tab[1,2])
    f1[i] <- 2*pre[i]*rec[i]/(pre[i] + rec[i])
  }
  results$acc[s] <- mean(acc)
  results$pre[s] <- mean(pre)
  results$rec[s] <- mean(rec)
  results$f1[s] <- mean(f1)
}
```

CV 10-folds

top10 f1-score

```
results |> arrange(desc(f1), cost) |> head(10)
```

```
##         cost       acc       pre rec        f1
## 1  0.2691535 0.9368421 0.9818182 0.9 0.9383459
## 2  0.2754229 0.9368421 0.9818182 0.9 0.9383459
## 3  0.2818383 0.9368421 0.9818182 0.9 0.9383459
## 4  0.2884032 0.9368421 0.9818182 0.9 0.9383459
## 5  2.0417379 0.9368421 0.9818182 0.9 0.9383459
## 6  0.2951209 0.9315789 0.9718182 0.9 0.9336090
## 7  0.3019952 0.9315789 0.9718182 0.9 0.9336090
## 8  0.3090295 0.9315789 0.9718182 0.9 0.9336090
## 9  0.3162278 0.9315789 0.9718182 0.9 0.9336090
## 10 0.3235937 0.9315789 0.9718182 0.9 0.9336090
```

best cost ≈ 0.27 and 2 (same performance)

select cost ≈ 0.27 to avoid “overfit”

best_model :

```
best_tune <- results |> arrange(desc(f1), cost) |> head(1)
best_tune |> select(cost)
```

```
##        cost
## 1 0.2691535
```

```
best_tune |>
  mutate(acc = paste0(round(acc * 100, 1), "%"),
  pre = paste0(round(pre * 100, 1), "%"),
  rec = paste0(round(rec * 100, 1), "%"),
  f1 = paste0(round(f1 * 100, 1), "%")) |>
    select(-cost)
```

```
##     acc   pre rec    f1
## 1 93.7% 98.2% 90% 93.8%
```

ประสิทธิภาพ (10-Folds)

Accuracy	= 93.7%
Precision	= 98.2%
Recall	= 90%
F1-Score	= 93.8%

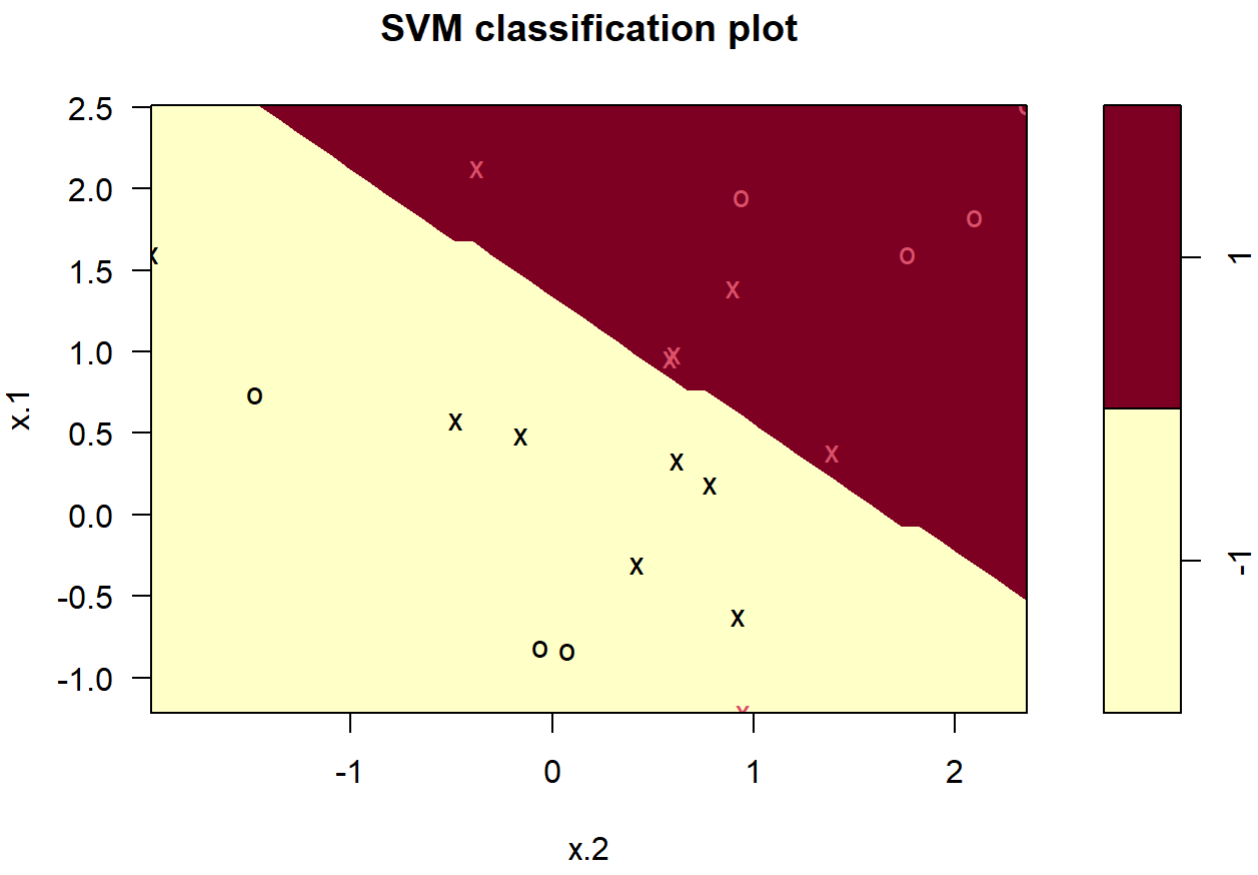
เลือก cost ≈ 0.27 และ run บน full College data ได้

```
fin_mod <- svm(y ~., data = dat, kernel = "linear", cost = best_tune$cost)

summary(fin_mod)
```

```
##
## Call:
## svm(formula = y ~ ., data = dat, kernel = "linear", cost = best_tune$cost)
##
##
## Parameters:
##    SVM-Type:  C-classification
##  SVM-Kernel:  linear
##        cost:  0.2691535
##
## Number of Support Vectors:  13
##
##  ( 7 6 )
##
##
## Number of Classes:  2
##
## Levels:
##  -1 1
```

```
plot(fin_mod, data = dat)
```



3.) Hw college grad rate class >70 = high , <=70 =

Low

```
library(dplyr)

college <- read.csv("College.csv")
college$label <- ifelse(college$Grad.Rate >70,"high","low")
college$label <- factor(college$label, levels = c("low","high"), labels = c(0, 1))
table(college$label)
```

```
##
##   0   1
## 477 300
```

```
str(college)
```

```
## 'data.frame':    777 obs. of  20 variables:
##  $ X          : chr  "Abilene Christian University" "Adelphi University" "Adrian College"
"Agnes Scott College" ...
##  $ Private    : chr  "Yes" "Yes" "Yes" "Yes" ...
##  $ Apps       : int  1660 2186 1428 417 193 587 353 1899 1038 582 ...
##  $ Accept     : int  1232 1924 1097 349 146 479 340 1720 839 498 ...
##  $ Enroll     : int  721 512 336 137 55 158 103 489 227 172 ...
##  $ Top10perc  : int  23 16 22 60 16 38 17 37 30 21 ...
##  $ Top25perc  : int  52 29 50 89 44 62 45 68 63 44 ...
##  $ F.Undergrad: int  2885 2683 1036 510 249 678 416 1594 973 799 ...
##  $ P.Undergrad: int  537 1227 99 63 869 41 230 32 306 78 ...
##  $ Outstate   : int  7440 12280 11250 12960 7560 13500 13290 13868 15595 10468 ...
##  $ Room.Board : int  3300 6450 3750 5450 4120 3335 5720 4826 4400 3380 ...
##  $ Books      : int  450 750 400 450 800 500 500 450 300 660 ...
##  $ Personal   : int  2200 1500 1165 875 1500 675 1500 850 500 1800 ...
##  $ PhD        : int  70 29 53 92 76 67 90 89 79 40 ...
##  $ Terminal   : int  78 30 66 97 72 73 93 100 84 41 ...
##  $ S.F.Ratio  : num  18.1 12.2 12.9 7.7 11.9 9.4 11.5 13.7 11.3 11.5 ...
##  $ perc.alumni: int  12 16 30 37 2 11 26 37 23 15 ...
##  $ Expend     : int  7041 10527 8735 19016 10922 9727 8861 11487 11644 8991 ...
##  $ Grad.Rate  : int  60 56 54 59 15 55 63 73 80 52 ...
##  $ label      : Factor w/ 2 levels "0","1": 1 1 1 1 1 1 1 2 2 1 ...
```

```
K=10
set.seed(123)
folds=sample(1:K,nrow(dat),replace=TRUE)

acc <- 1:K
pre <- 1:K
rec <- 1:K
f1 <- 1:K
tunecost <-  0.1^c(seq(-1, 1, by = 0.01))
results <- data.frame(
  cost = cost_tune,
  acc = NA,
  pre = NA,
  rec = NA,
  f1 = NA
)

dat <- college |> select(-X, -Grad.Rate)

dat_mod <- svm(label~., data = dat)
table(predict(dat_mod), college$label)
```

```
##
##       0   1
##   0 422  96
##   1  55 204
```

```
for (s in 1:length(cost_tune)){
  for (i in 1:K){
    traindat <- folds!=i
    svmfit <- svm(label ~ .,
                  data = dat[traindat,],
                  kernel = "linear",
                  cost = tunecost[s])

    ypred <- predict(svmfit, dat[-traindat,])
    mod_tab <- table(predict = ypred, truth = dat[-traindat,]$label)

    acc[i] <-  (mod_tab[1,1] + mod_tab[2,2])/((mod_tab[1,1] + mod_tab[2,2]) + (mod_tab[2,1] +
mod_tab[1,2]))
    pre[i] <- mod_tab[2,2]/(mod_tab[2,2] + mod_tab[2,1])
    rec[i] <- mod_tab[2,2]/(mod_tab[2,2] + mod_tab[1,2])
    f1[i] <- 2*pre[i]*rec[i]/(pre[i] + rec[i])
  }
  results$acc[s] <- mean(acc)
  results$pre[s] <- mean(pre)
  results$rec[s] <- mean(rec)
  results$f1[s] <- mean(f1)
}
```

CV 10 folds

top10 f1-score

```
results |> arrange(desc(f1), cost) |> head(10)
```

```
##         cost       acc       pre       rec        f1
## 1  0.2884032 0.7774485 0.7457061 0.6440000 0.6910181
## 2  0.2951209 0.7774485 0.7457054 0.6440000 0.6910162
## 3  0.3019952 0.7774485 0.7457054 0.6440000 0.6910162
## 4  0.2187762 0.7779639 0.7478197 0.6423333 0.6909845
## 5  0.3090295 0.7773196 0.7456082 0.6436667 0.6907853
## 6  0.2344229 0.7774485 0.7464657 0.6426667 0.6906101
## 7  0.2238721 0.7775773 0.7471467 0.6420000 0.6905119
## 8  0.2137962 0.7777062 0.7478305 0.6413333 0.6904118
## 9  0.3388442 0.7770619 0.7454196 0.6430000 0.6903359
## 10 0.2754229 0.7771907 0.7458816 0.6426667 0.6903287
```

Best Cost ≈ 0.29

Best Model :

```
college_best_tune <- results |> arrange(desc(f1), cost) |> head(1)
college_best_tune |> select(cost)
```

```
##        cost
## 1 0.2884032
```

```
college_best_tune |>
  mutate(acc = paste0(round(acc * 100, 1), "%"),
  pre = paste0(round(pre * 100, 1), "%"),
  rec = paste0(round(rec * 100, 1), "%"),
  f1 = paste0(round(f1 * 100, 1), "%")) |>
    select(-cost)
```

```
##     acc   pre   rec    f1
## 1 77.7% 74.6% 64.4% 69.1%
```

Accuracy	= 77.7%
Precision	= 74.6%
Recall	= 64.4%
F1-Score	= 69.1%

เลือก cost ≈ 0.29 และ run บน full College data ได้

```
fin_col_mod <- svm(label ~ .,
                   data = dat,
                   kernel = "linear",
                   cost = college_best_tune |> pull(cost))

summary(fin_col_mod)
```

```
##
## Call:
## svm(formula = label ~ ., data = dat, kernel = "linear", cost = pull(college_best_tune,
##     cost))
##
##
## Parameters:
##    SVM-Type:  C-classification
##  SVM-Kernel:  linear
##        cost:  0.2884032
##
## Number of Support Vectors:  416
##
##  ( 211 205 )
##
##
## Number of Classes:  2
##
## Levels:
##  0 1
```